



DairyDesk

Setup & Developer Guide — Windows and macOS

Audience Everyone on the team — new joiners start at section 1

Stack Django 5.1 + DRF · React 19 + Vite 8 · PostgreSQL 16

Runs on Docker (Python 3.13 / Node 22 pinned by the images)

Covers Machine setup, branch rules, migrations, coding rules

Also see `README.md` for a short version, `PROJECT_SPEC.md` for the spec

1. Set up your machine

You only need **two** programs: **Git** and **Docker Desktop**. You do **not** need to install Python, Node, or PostgreSQL — they live inside the containers, at the exact versions everyone else is using. That is the whole point: the app cannot break because of what is or isn't installed on your laptop.

Find your operating system below, do those steps once, then continue to section 1.3 which is the same for everybody.

1.1 WINDOWS 10 / 11

1 Turn on virtualisation (do this first)

Docker on Windows runs Linux inside a lightweight virtual machine. If virtualisation is off, Docker Desktop installs fine but

never starts

, with no clear error. Check it first. Open PowerShell and run:

```
wsl --status
```

If you see "*WSL2 is unable to start since virtualization is not enabled*", open PowerShell

as Administrator

(right-click → Run as administrator) and run:

```
dism /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart  
wsl --install --no-distribution
```

Then restart your computer.

This is required — `wsl --status` keeps showing the same error until you reboot, even after the feature is enabled.

Still failing after the reboot? Virtualisation is switched off in your BIOS/UEFI. Restart, enter BIOS setup (usually F2, F10 or Del), and enable

Intel VT-x

or

AMD SVM

.

2 Install Git

Download from

git-scm.com/downloads

and accept the defaults. This also gives you Git Bash, which is handy but optional — PowerShell works fine for everything here.

3 Install Docker Desktop

Download from

docker.com/products/docker-desktop

. Keep the

"Use WSL 2 instead of Hyper-V"

option ticked. Reboot if it asks.

4 Start Docker Desktop and wait

Launch it from the Start menu. The whale icon in the system tray animates while starting. Wait until it stops. Docker is not usable until then.

5 Check it worked

```
git --version
docker --version
docker compose version
```

All three must print a version number. If `docker` errors with *"daemon not running"*, Docker Desktop is not started yet.

WINDOWS NOTES

You no longer need to touch PowerShell's execution policy. That was only required for activating a Python virtualenv, and there is no virtualenv any more.

Run commands from **PowerShell** or **Git Bash** — both work. Just don't run the same stack from two terminals at once.

1.2 macOS (Apple Silicon or Intel)

1 Install Git

macOS ships with it. Run `git --version` in Terminal; if it is missing, macOS offers to install the Command Line Tools — accept. Or use `brew install git`.

2 Install Docker Desktop — pick the right build

Download from

docker.com/products/docker-desktop

and choose the download matching your chip:

- **Apple Silicon**
(M1/M2/M3/M4) — "Mac with Apple chip"
- **Intel**
— "Mac with Intel chip"

Not sure? Apple menu → About This Mac. "Chip: Apple M..." means Apple Silicon.

Open the `.dmg` and drag Docker into Applications.

3 Launch Docker and grant permission

Open Docker from Applications. macOS asks for your password the first time so it can install its helper — this is expected. Wait for the whale icon in the menu bar to stop animating.

4 Check it worked

```
git --version
docker --version
docker compose version
```

APPLE SILICON WORKS NATIVELY — NO ROSETTA NEEDED

All three base images (`python:3.13-slim` , `node:22-alpine` , `postgres:16`) publish native `arm64` builds, and the backend image has been test-built for `arm64` successfully. Nothing needs emulation, and you should not enable "Rosetta for x86/amd64 emulation" for this project.

1.3 RUN THE APP — same on both

You will be added to the private repo first, so make sure you are signed in to GitHub. Then:

```
git clone https://github.com/TanmayKamble004/DairyDesk.git
cd DairyDesk
docker compose up --build
```

That is the entire setup. **No `.env` file to copy, no `virtualenv`, no `npm install`**. Every setting has a working default built in.

The first run takes a few minutes to download and build the images (~260 MB backend, ~690 MB frontend). After that, starting takes seconds. You are ready when the logs show the demo data summary and then Vite's banner:

```
dairydesk_backend | Demo data seeded.
dairydesk_backend | Products:      7
dairydesk_backend | Starting development server at http://0.0.0.0:8000/
dairydesk_frontend | VITE v8.1.3  ready in 585 ms
```

What	Open this
The app	<code>http://localhost:5173</code>
API	<code>http://localhost:8000</code>
Django admin	<code>http://localhost:8000/admin</code>
Health check	<code>http://localhost:8000/api/health/</code>

Log in with one of the two demo accounts:

Role	Username	Password
Owner — full access + admin	<code>owner</code>	<code>owner123</code>
Staff — no financial data	<code>staff</code>	<code>staff123</code>

To stop: press `Ctrl+C` in that terminal. Next time, just `docker compose up` — you only need `--build` after someone adds a new dependency.

IF SETUP FAILS, CHECK THESE THREE FIRST

"Docker daemon not running" — Docker Desktop isn't started, or on Windows, virtualisation is still off. Go back to step 1.1.

"port is already allocated" — something else uses 8000, 5173 or 5432. Close it, or create a `.env` file and set `BACKEND_PORT`, `FRONTEND_PORT` or `POSTGRES_PORT` to a free number.

`exec /app/docker-entrypoint.sh: no such file or directory` — your Git converted the file's line endings. Delete the folder and clone again; `.gitattributes` in the repo prevents this on a fresh clone.

2. Understand the domain before you change it

DairyDesk is a small codebase with a few sharp edges. Almost every bug introduced here comes from missing one of the rules in section 6 — they are not style preferences, they are correctness requirements around **money**, **stock** and **who is allowed to see what**.

Four models carry all the business logic. Skim them before touching anything:

Model	What it means — and the part people get wrong
Product	A sellable item with a <i>current</i> <code>selling_price</code> . Changing this price must never change past orders.
StockBatch	A physical delivery with a quantity and an expiry date. Stock lives in batches, never on the product.
Order / OrderItem	An order deducts stock at creation time and <i>snapshots</i> the price into <code>OrderItem.unit_price</code> .
Invoice	Created automatically when an order becomes <code>delivered</code> . Owner-only data.

DON'T RUN THINGS TWICE

If you previously ran this project with a local virtualenv and `npm run dev`, don't do both. Two backends fighting over port 8000 produces very confusing errors.

3. Branch and pull request rules

Nobody commits straight to `main`. You work on your own branch, push it, and open a pull request. The project owner tests it and merges. This keeps `main` always working, so nobody's laptop breaks because of someone else's half-finished change.

Rule 1 — Never commit to `main`

Not even a small fix. Not even a typo. Always make a branch.

Rule 2 — Always start from the latest `main`

If you branch from old code, your pull request will conflict with things other people already merged. Get the newest code first, every time:

```
git checkout main
git pull           # get everyone else's latest work
git checkout -b your-name # make your branch (e.g. tvk, riya, amit)
```

Rule 3 — One branch per feature

Do not put three unrelated changes on one branch. Small branches get reviewed and merged fast. Big mixed branches sit for days and cause conflicts.

Rule 4 — Commit your work, then push your branch

```
git add .
git status           # LOOK at this before committing
git commit -m "add customer search on orders page"
git push -u origin your-name # first push only; later just: git push
```

Write the message as what the change *does*. "add customer search" is good. "update", "fix", "changes" tell the next person nothing.

Rule 5 — Open a pull request on GitHub

Go to the repo on GitHub. It will show a banner for your just-pushed branch with a **Compare & pull request** button. Base branch is `main`, compare branch is yours. Write one or two lines about what you changed and how you tested it. Then wait — **do not merge your own pull request**.

Rule 6 — Test it yourself before asking for review

Run the app and click through your change first. Log in as **both** `owner` and `staff`. A pull request that was never run wastes the reviewer's time.

Rule 7 — After your branch is merged, delete it and start fresh

```
git checkout main
git pull           # now includes your merged work
git branch -d your-name # delete the old local branch
```

Next feature, make a new branch from the updated `main`. Do not keep reusing one old branch forever.

NEVER DO THESE

`git push --force` on a shared branch — it deletes other people's commits.

Committing `.env`, `node_modules/`, `.venv/` or `__pycache__`. `.gitignore` already blocks them. Do not force them in.

Merging your own pull request without anyone testing it.

For the reviewer: test the branch safely

Testing someone's branch on your normal stack can leave their database changes in *your* database, even if you reject the pull request. Use a separate throwaway stack instead — `-p dairydesk_pr` gives it its own database:

```
git fetch origin
git switch their-branch-name

docker compose down                # stop your normal stack, keeps your data
docker compose -p dairydesk_pr up --build # isolated test stack, fresh database

# ...test the app at localhost:5173 as owner AND staff...

docker compose -p dairydesk_pr down -v # delete the throwaway database
git switch main
docker compose up -d                  # back to your own work, untouched
```

On GitHub, use **Squash and merge** so each feature becomes one clean commit on `main`.

4. Migrations — the thing that breaks everyone's app

What a migration actually is

Your models are Python code in `backend/core/models.py`. But the real data lives in PostgreSQL, which needs actual tables and columns. A **migration** is a generated file that tells the database how to change so it matches your models.

Think of it as the instruction slip: "add a column called `is_active` to the products table." Django keeps a list of which slips have already been carried out.

Right now the project has exactly one: `backend/core/migrations/0001_initial.py`. It created all nine `core_*` tables.

When you need one

Only when you change `models.py`. Adding a field, removing a field, renaming something, changing a model's options. Editing views, serializers, or any frontend file needs no migration at all.

The three steps — never skip step 3

1 Change the model

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    is_active = models.BooleanField(default=True) # you added this
```

2 Generate the migration file

```
docker compose exec backend python manage.py makemigrations
```

This creates something like `core/migrations/0002_product_is_active.py` in your project folder.

3 Commit that file together with your model change

```
git add backend/core/models.py backend/core/migrations/0002_product_is_active.py
git commit -m "add is_active flag to products"
```

THE #1 MISTAKE IN THIS PROJECT

Changing the model but **forgetting to commit the migration file**. Your app works because your database was already updated. Everyone else pulls your code and gets:

```
ProgrammingError: column core_product.is_active does not exist
```

Their code expects a column their database has never been told about. Before pushing, always run `git status` and check the migration file is included.

Applying migrations

You almost never run `migrate` by hand. The backend container runs it automatically every time it starts. So after pulling someone's work:

```
git pull
docker compose restart backend # applies any new migrations
```


Check what your database currently has:

```
docker compose exec backend python manage.py showmigrations core
```

[X] means applied, [] means not yet.

Migration rules

- **One migration per model change, committed with it.** Never separately, never later.
- **Never edit a migration file that is already on `main`.** Other people have already run it. Their database will not re-run it, so your edit silently does nothing for them. Make a new migration instead.
- **Never delete a migration file that is on `main`,** for the same reason.
- **Never hand-write SQL to change tables.** Django would not know about it and the next `makemigrations` would fight you.
- **Two people adding migrations at once** can both create a `0002_`. Whoever merges second should pull `main`, delete their own file, and re-run `makemigrations` so it becomes `0003_`.
- **If a new field must have a value for existing rows,** give it a `default=`, or Django will stop and ask you awkward questions.

SAFE ESCAPE HATCH

If your local database gets into a confusing state while experimenting, throw it away and start clean — it only holds demo data:

```
docker compose down -v && docker compose up
```

You get a fresh database with all migrations applied and demo data seeded.

5. Pre-flight checklist

Run through this before opening a pull request. It takes two minutes and catches nearly everything.

Any change

- ☐ The app still starts clean: `docker compose down && docker compose up` with no errors in the logs.
- ☐ You logged in as **both** `owner` and `staff` and clicked through the affected pages.
- ☐ No secrets, API keys, or absolute URLs hardcoded — config comes from environment variables only.
- ☐ No `console.log` / `print()` debugging left behind.
- ☐ `git status` is clean of junk — no `.env`, no `__pycache__`, no `node_modules`.
- ☐ You are on your own branch, not `main`.

If you touched a model

- ☐ Generated the migration: `docker compose exec backend python manage.py makemigrations`
- ☐ **Committed the migration file.** Forgetting it breaks every teammate's database.
- ☐ Confirmed it applies: `docker compose restart backend` shows `Applying ... OK`.
- ☐ Checked nothing is pending: `makemigrations --check --dry-run` prints *No changes detected*.
- ☐ Updated `seed_demo.py` if the new field is needed for a realistic demo.

If you added or changed an API endpoint

- ☐ Route registered in `backend/core/urls.py` (router for viewsets, `path()` for APIViews).
- ☐ New fields added to the serializer's explicit `fields` list — they will not appear otherwise.
- ☐ Decided the permission deliberately. The global default is `IsAuthenticated` ; financial data needs `IsOwner` .
- ☐ Tested it while logged in as `staff` , not just `owner` .

If you touched the frontend

- ☐ Used the shared `api` client, not raw `axios` or `fetch` .
- ☐ Reused components from `components/ui.jsx` instead of restyling from scratch.
- ☐ Errors surfaced through `apiErrorMessage(error)` .
- ☐ Owner-only pages wrapped in `<RequireOwner>` **and** protected server-side.

6. The rules that actually matter

These encode business correctness. Breaking one produces wrong money, wrong stock, or a data leak — usually with no error message.

6.1 Role gating lives in two places. Always do both.

Financial data is owner-only. That is enforced on the server by `IsOwner` (`core/permissions.py`) and in the UI by `RequireOwner` (`App.jsx`).

Why: the frontend guard is cosmetic. A staff user who calls the API directly gets the data anyway unless the server refuses. Conversely, server-only protection gives staff a broken page instead of a hidden one. **Both, every time.**

6.2 Dashboard KPIs are filtered by role in the view

`DashboardView` only puts `total_available_stock_value`, `todays_sales_total` and `unpaid_invoice_count` in the response when `is_owner`. The frontend renders whichever keys are present.

Why: adding a money KPI outside the `if is_owner` block silently leaks revenue data to staff, and nothing in the UI will look wrong.

6.3 `unit_price` is a historical snapshot — never recompute it

`OrderItem.unit_price` is copied from the product at order-creation time. Totals must always be derived from it, never from `product.selling_price`.

Why: if you read the live price, raising a product's price retroactively rewrites every historical invoice. Use `item.line_total`.

6.4 Stock deduction is FIFO and must run inside a transaction

`deduct_stock_fifo()` in `core/services.py` takes from the oldest non-expired batches first and uses `select_for_update()`. Every caller must wrap it in `transaction.atomic()` — see `OrderSerializer.create`.

Why: the row locks only hold inside a transaction. Outside one, two simultaneous orders can sell the same batch twice. It also validates all-or-nothing: if any product is short, it raises and deducts nothing.

6.5 Order status moves one way only

`pending → processed → delivered`, enforced by `ORDER_STATUS_TRANSITIONS`. `status` is read-only on `OrderSerializer`; changes go through `PATCH /api/orders/{id}/` only.

Why: reaching `delivered` creates an invoice. Allowing backward moves would orphan invoices and corrupt the sales figures.

6.6 Delivering an order auto-creates its invoice

`ensure_invoice(order)` is idempotent (`get_or_create`). Never create `Invoice` objects by hand elsewhere.

Why: one order has exactly one invoice (`OneToOneField`), and the total must be computed the same way everywhere.

6.7 `expiry_status` is computed, never stored

It is a `@property` on `StockBatch` derived from today's date and `AGEING_THRESHOLD_DAYS = 3`. Fresh > 3 days left, ageing ≤ 3 days, expired past.

Why: a stored column would be stale the next morning. This one constant drives the 3D shelf colours, the dashboard warning counts and the inventory endpoint — change it and all three move together.

6.8 Expired stock counts as visible, not as available

Across `InventoryView`, `DashboardView` and `Product.available_quantity`: expired batches are shown and counted, but excluded from available quantity and stock value. Batches with `quantity == 0` are skipped entirely.

Why: the shop still physically holds expired stock and needs to see it, but must never sell it or count it as money.

6.9 Use the custom user model

`AUTH_USER_MODEL = "core.User"`, with a `role` field of `owner` / `staff`. Reference it via `get_user_model()` or `settings.AUTH_USER_MODEL`.

Why: importing `django.contrib.auth.models.User` silently targets the wrong table.

6.10 Deletions are deliberately blocked

`Customer → Order` and `Product → OrderItem` use `on_delete=PROTECT`.

Why: deleting a customer or product that appears in order history would destroy the audit trail. If a "delete" feature is requested, add a deactivation flag instead of weakening these.

7. Backend conventions

Where	Convention
<code>models.py</code>	Business <i>data</i> . Derived values are <code>@property</code> , not columns.
<code>services.py</code>	Business <i>rules</i> shared by more than one caller (stock deduction, invoice creation). Put logic here, not in views.
<code>serializers.py</code>	Validation and write logic. Multi-step writes go in <code>create()</code> / <code>update()</code> wrapped in <code>transaction.atomic()</code> .
<code>views.py</code>	Thin. Wire up querysets, permissions and serializers — delegate real work to services.
<code>permissions.py</code>	Reusable permission classes. Add here rather than checking roles inline in views.

Things to keep doing

- **Explicit serializer `fields` lists.** Never `fields = "__all__"` — it exposes new columns the moment they are added.
- `select_related` / `prefetch_related` **on every list queryset.** The existing viewsets all do this; copy the pattern or you will add N+1 queries.
- **Money is `DecimalField`, never `float`.** Serialize totals with `str()`, as the dashboard does.
- **Dates via `timezone.localtime()`, not `date.today()`** — `USE_TZ` is on.
- **Validation errors as DRF `ValidationError`** with a field key, so the frontend's `apiErrorMessage` can render them.

8. Frontend conventions

- **Always import `api` from `src/api/client.js`.** It attaches the JWT and transparently refreshes it on a 401 (single-flight, so parallel calls refresh once). A raw `fetch` or bare `axios` call bypasses both and will 401 randomly after eight hours.
- **Reuse `components/ui.jsx`** — `Card`, `PageHeader`, `Spinner`, `ErrorAlert`, `Badge`, `Th`, `Td`, plus the `inputClass` / `buttonPrimary` / `buttonSecondary` class strings.
- **Every page handles three states:** loading, error, loaded. Copy the pattern from `pages/Inventory.jsx`.
- **Format currency with the existing `formatINR` helper** (`Dashboard.jsx`) rather than inventing another one.
- **Tailwind utility classes only.** No new CSS files — Tailwind 4 is wired in through the Vite plugin.
- **Never read tokens from `localStorage` directly.** Use `getAuth()` / `useAuth()`.

THE 3D SHELF IS PERFORMANCE-SENSITIVE

`InventoryShelf.jsx` renders with react-three-fiber. Objects created inside the render path (geometries, materials, arrays) get rebuilt every frame and will tank the frame rate. Memoise them, and test with the full seeded dataset rather than two products.

9. Working with Docker day to day

Your files are mounted into the containers, so **editing** `.py`, `.jsx` or `.css` **needs no command** — Django reloads in about a second, Vite hot-swaps the module in the browser. Only these need action:

Situation	Command
Added a Python package	<code>docker compose up -d --build backend</code>
Added an npm package	<code>docker compose up -d --build --renew-anon-volumes frontend</code>
Changed a model	<code>docker compose exec backend python manage.py makemigrations</code>
Pulled new migrations	<code>docker compose restart backend</code>
Any <code>manage.py</code> command	<code>docker compose exec backend python manage.py <cmd></code>
Reading logs	<code>docker compose logs -f backend</code>
Shell in a container	<code>docker compose exec backend bash</code> (frontend is Alpine → use <code>sh</code>)

THREE COMMANDS THAT DESTROY DATA

`docker compose down -v` — the `-v` deletes the Postgres volume and everything in it. Plain `down` is the safe one.

`python manage.py seed_demo` — **clears** products, batches, customers, orders and invoices before reseeding. It is not additive.

`python manage.py flush` — wipes all rows. Rarely what you want here.

SAFE BY DESIGN

Demo data is seeded **only into an empty database**. Restarting containers, rebuilding images and running `docker compose down` all preserve your data.

Do not change these without understanding why

- `DB_HOST: db` and `DB_PORT: 5432` are **hardcoded in** `docker-compose.yml`, not read from `.env`. Inside the Docker network the backend reaches Postgres by service name on its container port; the `.env` values describe how your *host* reaches it and may differ (5433 is common when 5432 is taken).
- `.gitattributes` **forces LF endings on** `*.sh`. If `docker-entrypoint.sh` ever gets CRLF, the container dies with a baffling `no such file or directory`.
- `node_modules` **lives in a Docker volume**, deliberately masking the host copy (which may hold Windows binaries). That is why npm changes need `--renew-anon-volumes`.

10. Testing — read this honestly

THERE IS NO AUTOMATED TEST SUITE

`backend/core/tests.py` is still the empty Django stub, and there are no frontend tests. Nothing will catch a regression for you. Until that changes, **manual verification is mandatory**, and the burden is on the person making the change.

Minimum manual pass for a backend change

1. Log in as `owner` — dashboard, inventory, orders and invoices all load.
2. Log in as `staff` — financial KPIs are absent and `/invoices` is not reachable.
3. Create an order — stock drops from the oldest batch first.
4. Try to order more than is available — you get a clear error and **nothing** is deducted.
5. Move an order `pending → processed → delivered` — an invoice appears with the correct total.
6. Re-check the dashboard — today's order count and sales total moved as expected.

Reset to a known state whenever you need one:

```
docker compose exec backend python manage.py seed_demo
```

The seed deliberately includes fresh, ageing *and* expired batches so all three shelf colours are visible — keep it that way if you edit it.

ADDING TESTS IS WELCOME

The highest-value first tests are `deduct_stock_fifo` (ordering, shortage roll-back, concurrency), the order status transition rules, and a staff-cannot-read-invoices permission test. Run them with

```
docker compose exec backend python manage.py test .
```

11. Common mistakes

Symptom	Real cause
<code>column core_...</code> does not exist	Migration not generated, or generated but not committed.
Teammate's app broke after your merge	Model change committed without its migration file.
Your database has columns <code>main</code> doesn't know about	You tested someone's branch on your normal stack instead of <code>-p dairydesk_pr</code> .
Two migrations both numbered <code>0002_</code>	Two people branched at the same time. Second to merge re-runs <code>makemigrations</code> .
Your pull request has conflicts	You branched from stale code. Always <code>git pull</code> on <code>main</code> first.
Field added but missing from the API response	Not in the serializer's explicit <code>fields</code> list.
Staff can see money	Missing <code>IsOwner</code> server-side, or a KPI added outside the <code>is_owner</code> block.
Requests 401 after a few hours	Bypassed the shared <code>api</code> client, so the refresh interceptor never ran.
Old invoice totals changed by themselves	Reading <code>product.selling_price</code> instead of <code>OrderItem.unit_price</code> .
Stock went negative or double-sold	<code>deduct_stock_fifo</code> called outside <code>transaction.atomic()</code> .
<code>Cannot find module</code> after pulling	New npm package — rebuild with <code>--renew-anon-volumes</code> .
Port 8000 or 5173 already in use	A host-run <code>runserver / npm run dev</code> is still going, or set <code>BACKEND_PORT / FRONTEND_PORT</code> .
All your data vanished	<code>docker compose down -v</code> . The volume is gone; reseed.